

Day 3

Last Time

- ▶ ML Review
 - ▶ Types of Models
 - ▶ Model Assessment
 - ▶ Model Training
- ▶ Neural Networks

To Do

- ▶ Discussion 2 (Feb 7th)
- ▶ Quiz 1 Retakes

Today

- ▶ Neural Networks
- ▶ Numpy and PyTorch
(Broadcasting and Optimization)
- ▶ Quiz 1



The MSU Denver Math Seminar
Presents

"Fixed Points and Fair Division"

Dr. JOHN CARTER

**Monday, February 9, 2026
2:00 PM in Science 1015**

Join Us for Pizza, Soda, and a Good Talk!

Free Books for All Students and Faculty!
(Many thanks to Louis Krupp for donating books to our math seminar!)

Abstract:

Imagine a rental house with three rooms where the rent is \$900 per month. Three people plan to rent the house together, but the rooms are wildly different. What is the fairest way to divide the rent? If you immediately said 'Obviously \$300 each' imagine one roommate paying for a glorified closet while the other gets a private bath and a kitchenette. Does it still seem fair? If not is there a mathematical solution? I will share one solution that surprisingly uses the discrete version of the Brower fixed point theorem. I'll even pass on a link to a program that helps you solve such division problems.

Info about the speaker:

Dr. John Carter was born in Denver, later he joined the faculty at MSU Denver where he is a Professor and, some say, the chair of MAST. He studied Algebraic Topology at the University of Oregon and is always on the lookout for real world applications of topology.

<https://msudenver.enterprise.localist.com/event/2026-msu-denver-scholarship-fair>

2026 MSU Denver Scholarship Fair

Wednesday, February 11, 2026 10:00am to 2:00pm
MST

I'm Interested



Lab 1: Question 1

Q1

Go back to where we declared `top_n_to_remove` ([here](#)). Try changing the value to see if you can improve the accuracy of our model. No need to justify or explain the value you use, just try experimenting with considerably smaller or larger values and see if you can improve the accuracy.

What is the model accuracy that you get? Were you able to beat ~66% (the accuracy if every tweet were predicted to be negative)?

The model accuracy that I got was 69.76% with `top_n_to_remove = 450`

`I was able to get a model accuracy of 71.89% with a `top_n_to_remove` of 500.

Lowering the value below 100 resulted in worse accuracy whereas a higher the number on `n` gave higher accuracy (at least up until `n=500`, which gives 71.89% accuracy).

decreasing `top_n_to_remove` to 10 improved the accuracy to ~0.50
decreasing `top_n_to_remove` to 50 improved the accuracy to ~0.53
increasing `top_n_to_remove` to 400 improved the accuracy to ~0.67
increasing `top_n_to_remove` to 500 improved the accuracy to ~0.72
increasing `top_n_to_remove` to 600 improved the accuracy to ~0.73
increasing `top_n_to_remove` to 1000 improved the accuracy to ~0.81
increasing `top_n_to_remove` to 2000 improved the accuracy to ~0.85
increasing `top_n_to_remove` to 3000 improved the accuracy to ~0.87
increasing `top_n_to_remove` to 10000 improved the accuracy to ~0.83

Lab 1: Question 2

Q2

Does the model accuracy we found above give you a good idea of how this model will perform on future tweets? If not, then what could you do to get a better idea of how it will perform on future tweets?

train - test split
or
cross validation
or
k-fold validation

Lab 1: Question 3

Q3

In Q2 you were asked about the model accuracy we found earlier, and whether we could expect it to be representative of the accuracy we'd see on future, unseen, tweets. Hopefully you realized that without a proper train-test split, we won't have a good idea. Would right now (after we've created the classifier_tokens), be a good time to create a train-test split? Why or why not?

Yes right no would be a good time to create a train-test split because we ensure that the train-test sets are processed the same as well as avoiding data leakage.

It would be a good time to create a train-test split because the model can now be trained using the classifier tokens which will increase the accuracy of the model.

Yes! I would say that now is a good time for a train-test split. We've cleaned up the tokens and are now ready to train the model. Our data has been cleaned and processed all that's left is to split the data into training and testing sets.

Creating a train-test split after creating the classifier tokens will cause data leakage between the test data and the train data. This is because when training on the train data a model would have information about the contents and sentiments of the tokens for the test set from the classifier.

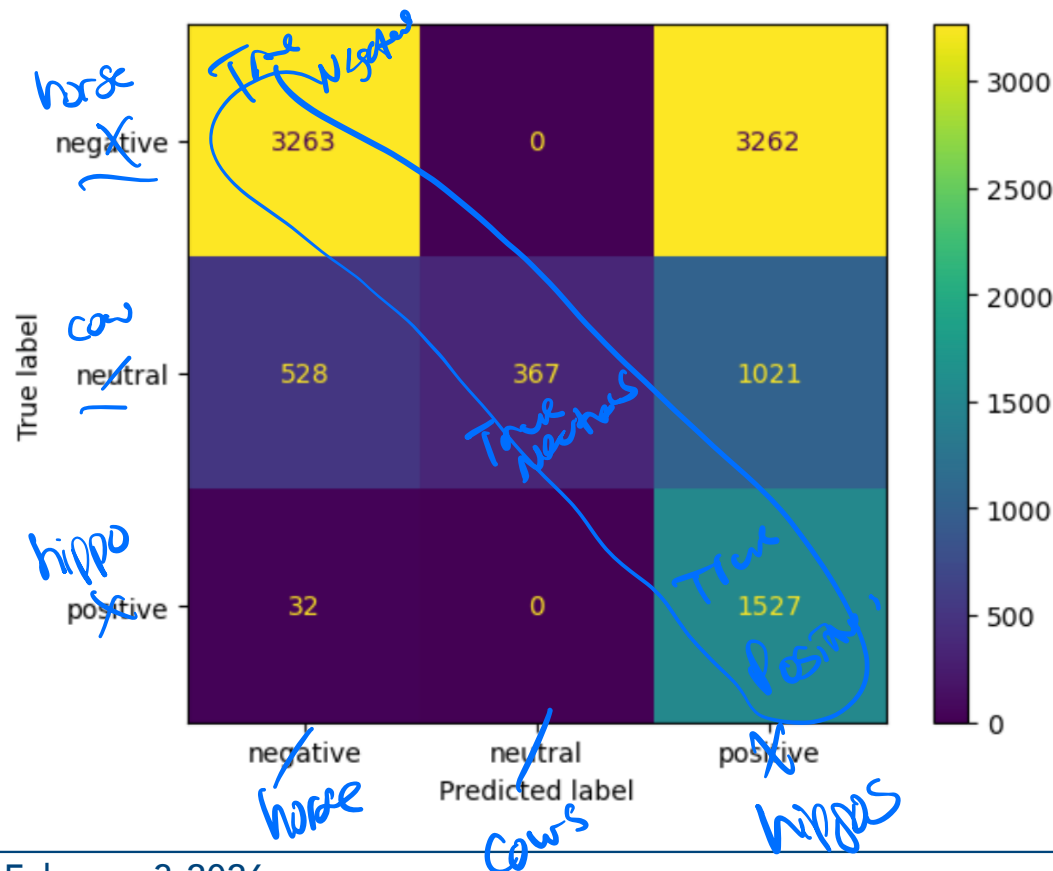
No, we need to split the data before creating these classifiers, otherwise our model has information about the test dataset.

(Recap) Model Assessment: Model Perf. Metrics

Our Solution: Overall accuracy was ~38% and we looked at accuracy by label with the Confusion Matrix.

```
from sklearn.metrics import ConfusionMatrixDisplay
disp = ConfusionMatrixDisplay(confusion_matrix(df['sentiment'], df['predicted_sentiment']),
disp.plot()
```

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7be6cbc5fc10>



Sum of diagonal
Sum of whole matrix

(Recap) Intro/Review: Machine Learning Models

Parametric models assume a specific functional form or distribution of the data ,e.g., linear or quadratic relationships (it summarizes the data through a fixed number of parameters, regardless of the size of the dataset)

- model structure is predefined
- training involves estimating values of a fixed number of parameters
- often requires assumptions about underlying distribution of data

One way to define/describe any parametric model (including neural networks) is as follows.

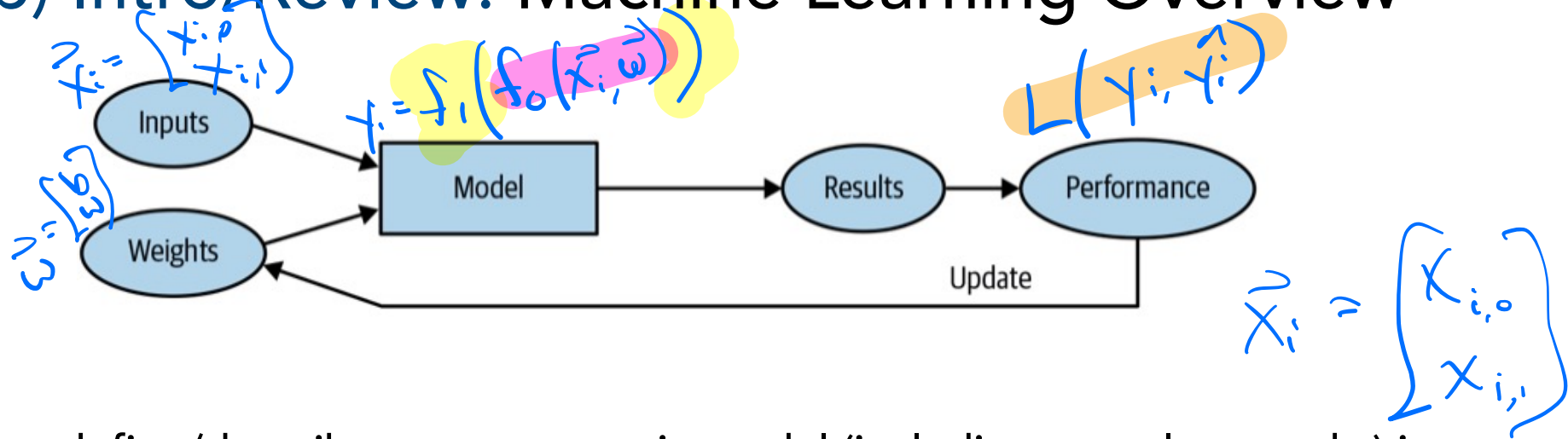
Given a dataset, D , consisting of n pairs of labels and features, (y_i, x_i) , a parametric model f with trainable parameters w can make predictions:

outcome / label *features*

$$\vec{w} = \begin{bmatrix} b \\ w \end{bmatrix}$$
$$\vec{x}_i = \begin{bmatrix} 1 \\ 1.75 \end{bmatrix}$$
$$\vec{w} \cdot \vec{x}_i = b \cdot x_0 + w \cdot x_1$$
$$\hat{y}_i = f(\vec{x}_i, \vec{w})$$

prediction *p-dimensional vector* *p-dimensional vector* *where p is # of features*

(Recap) Intro/Review: Machine Learning Overview



One way to define/describe any parametric model (including neural networks) is as follows.

$$u = f_0(\vec{x}_i; \vec{w}) = b \cdot x_{i,0} + \omega \cdot x_{i,1}$$

$$f_1(u) = \phi(u) = \frac{1}{1 + e^{-u}}$$

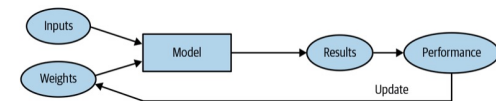
Given a dataset, D , consisting of n pairs of labels and features, (y_i, x_i) , a parametric model f with trainable parameters w can make predictions:

$$\hat{y}_i = f\left(\underset{p \times 1}{\vec{x}_i}, \underset{p \times 1}{\vec{w}}\right) = \phi\left(b \cdot x_{i,0} + \omega \cdot x_{i,1}\right)$$

Finding the value of w^* is done by minimizing a loss function, L :

$$\min_{w.r.t \vec{w}} L(\vec{w}) = \frac{1}{n} \sum_{i=1}^n (y_i - f(\vec{x}_i, \vec{w}))^2$$

(Recap) ML Intro/Review: Loss Functions



Purpose of loss function is to provide model with feedback based on the current values of the parameter (i.e. weights and biases)

- loss function should be a smooth function that is conducive to numerical (thus, it should vary even if parameter values change only a little bit)

Two general loss functions used, depending on whether it is a regression or classification problem:

Mean-square error: $L(\vec{w}) = \frac{1}{n} \sum_{i=1}^n \left(y_i - f(\vec{x}_i, \vec{w}) \right)^2$

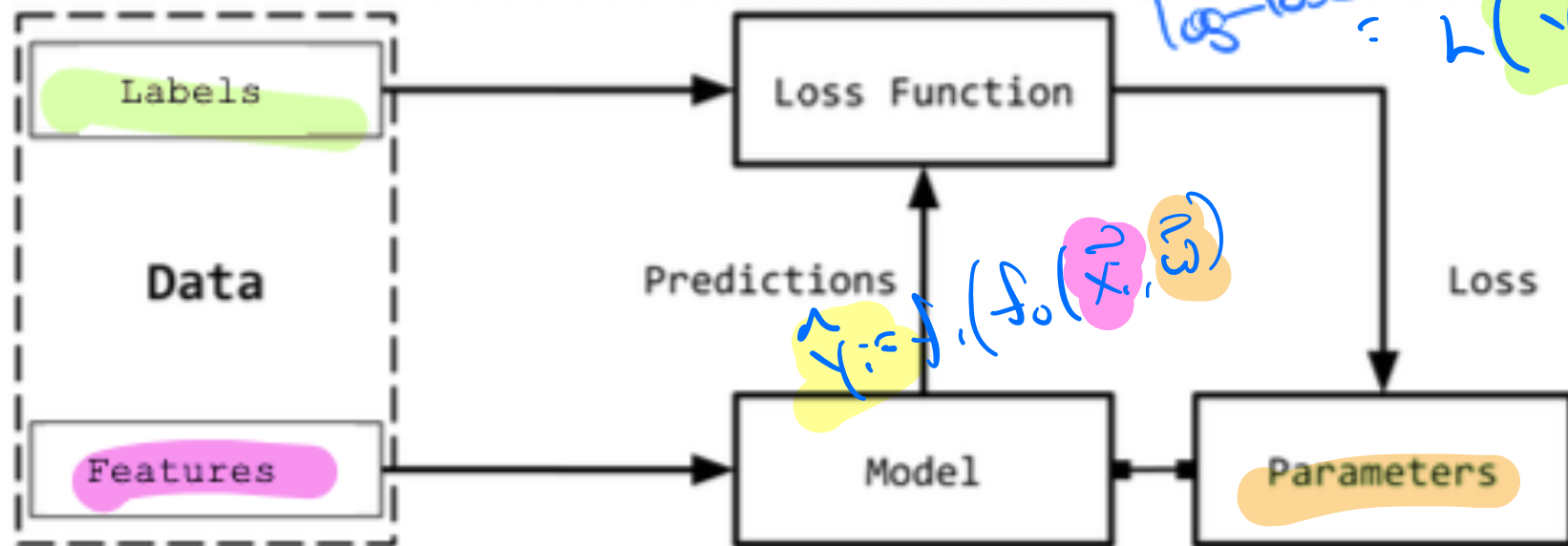
more for regression

Cross-entropy: $L(\vec{w}) = - \sum_{i=1}^n y_i \cdot \log \left(f(\vec{x}_i, \vec{w}) \right)$

(aka Log-loss)

more for classification

(Recap) ML Intro/Review: Supervised Learning



Q: What would each component above look like for...

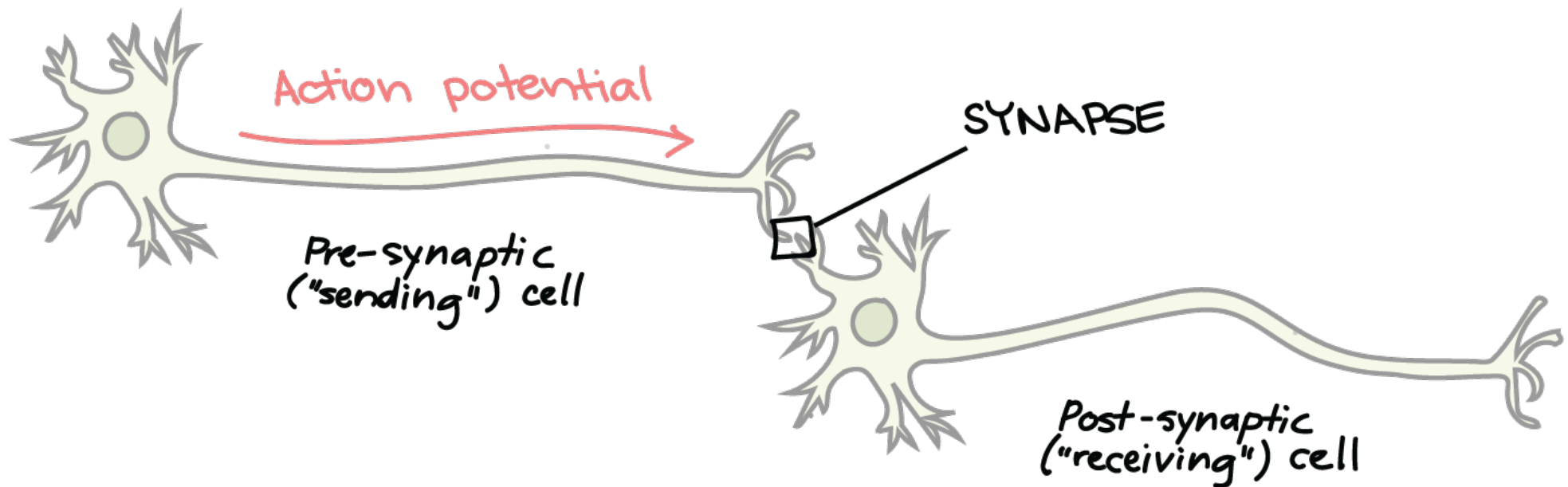
predicting survival of a Titanic passenger (yes/no) from, age, cabin number/type, ticket price, ...

Neural Networks

Early Machine Learning: Biological Inspiration

"When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic change takes place in one or both cells such that A's efficiency, as one of the cells firing B, is increased."

– Donald Hebb
(psychologist/neuropsychologist)

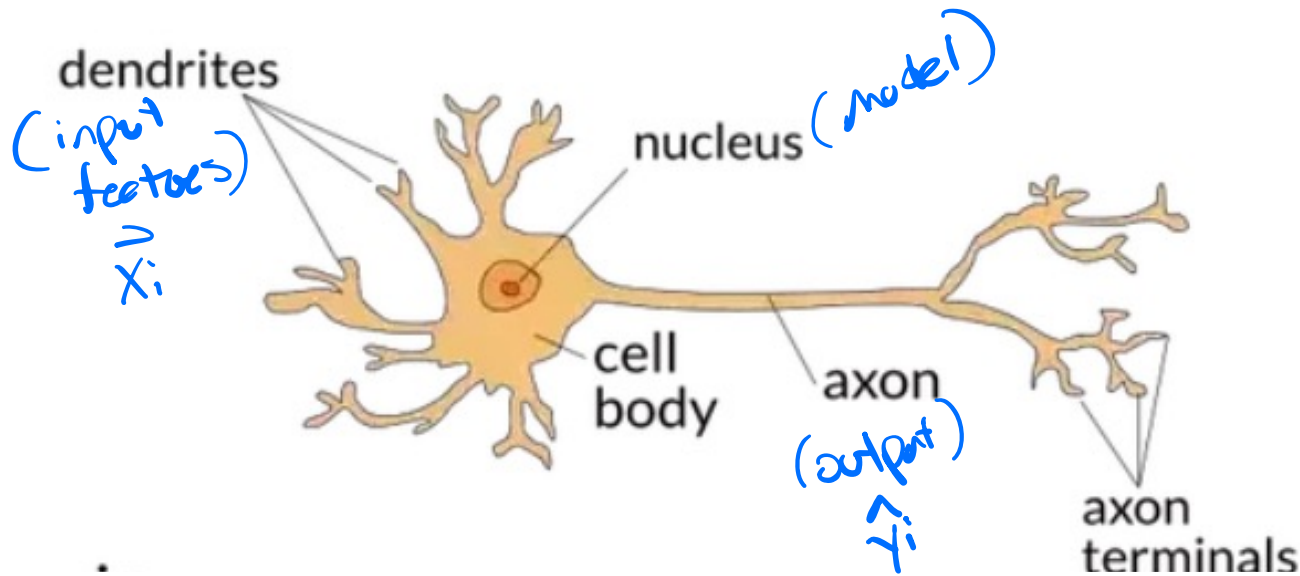


Early Machine Learning: Perceptron Model

Artificial Neural Network (ANN) – based on a biological neuron

- Proposed in 1943 by McCulloch and Pitts, *"A Logical Calculus of the Ideas Immanent in Nervous Activity"*:

"Because of the 'all-or-none' character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms."



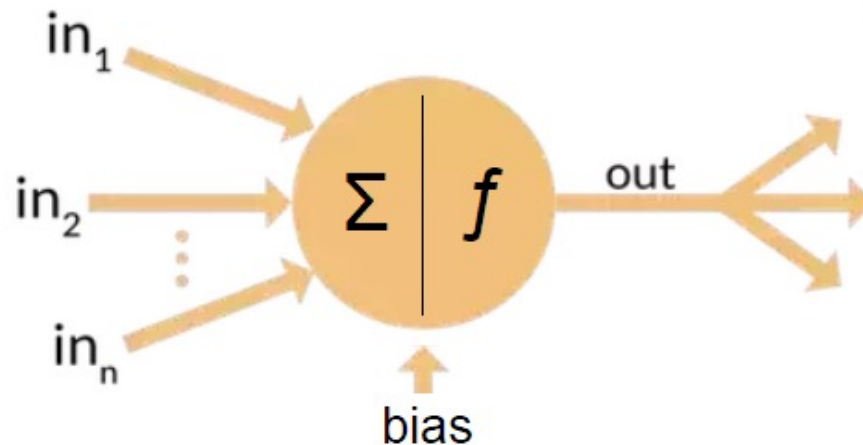
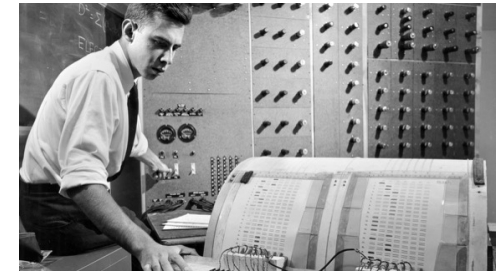
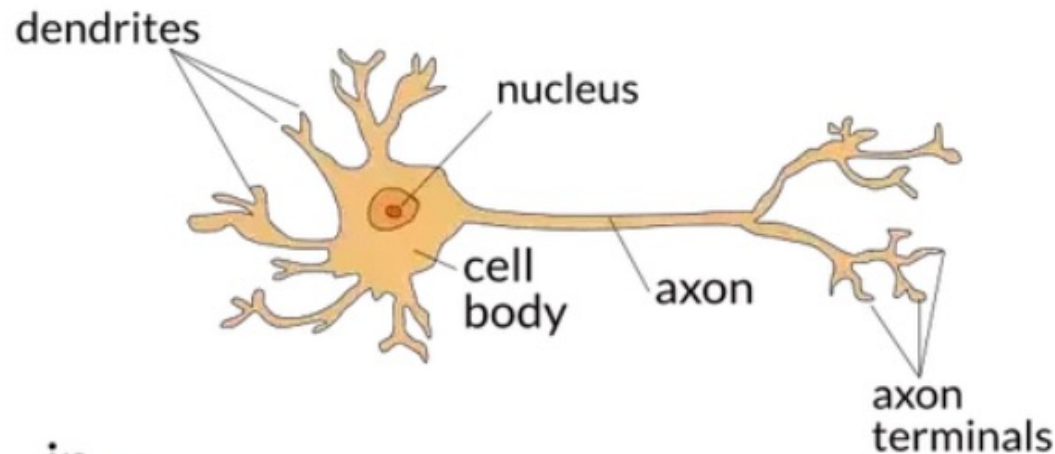
Early Machine Learning: Perceptron Model

Work later picked up by Frank Rosenblatt

- Mark I Perceptron (a working computer/device based on an ANN)

"We are about to witness the birth of such a machine – a machine capable of perceiving, recognizing and identifying its surroundings without any human training or control."

- Rosenblatt (1958)



Early Machine Learning: Coining the Term

Although used earlier, Arthur Samuel, popularized the term, "Machine Learning", and developed a self-learning checkers-playing program (was not based on a neural network though!)

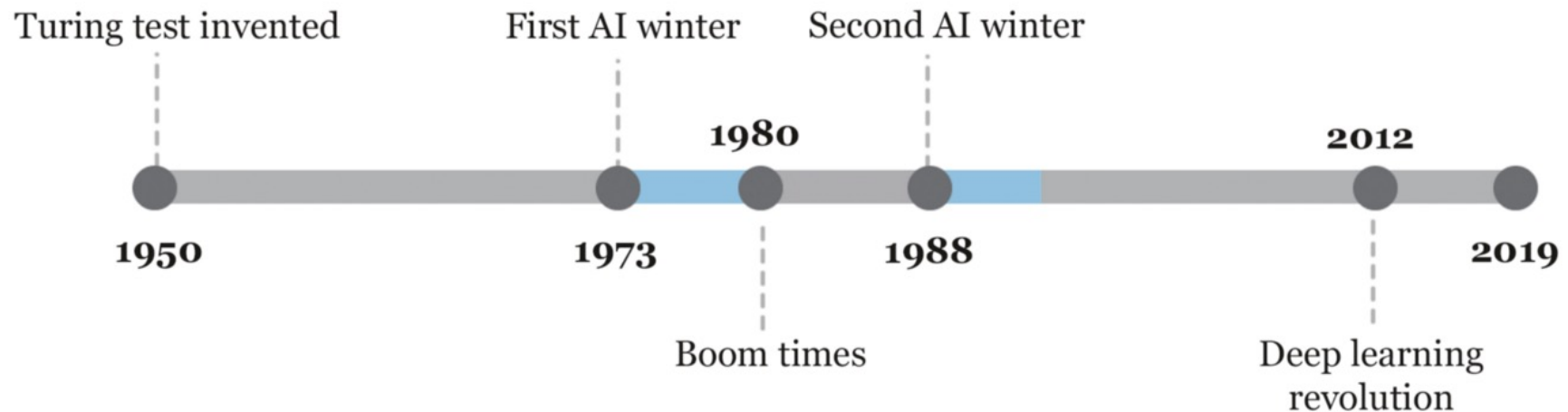


Arthur Lee Samuel (1959)

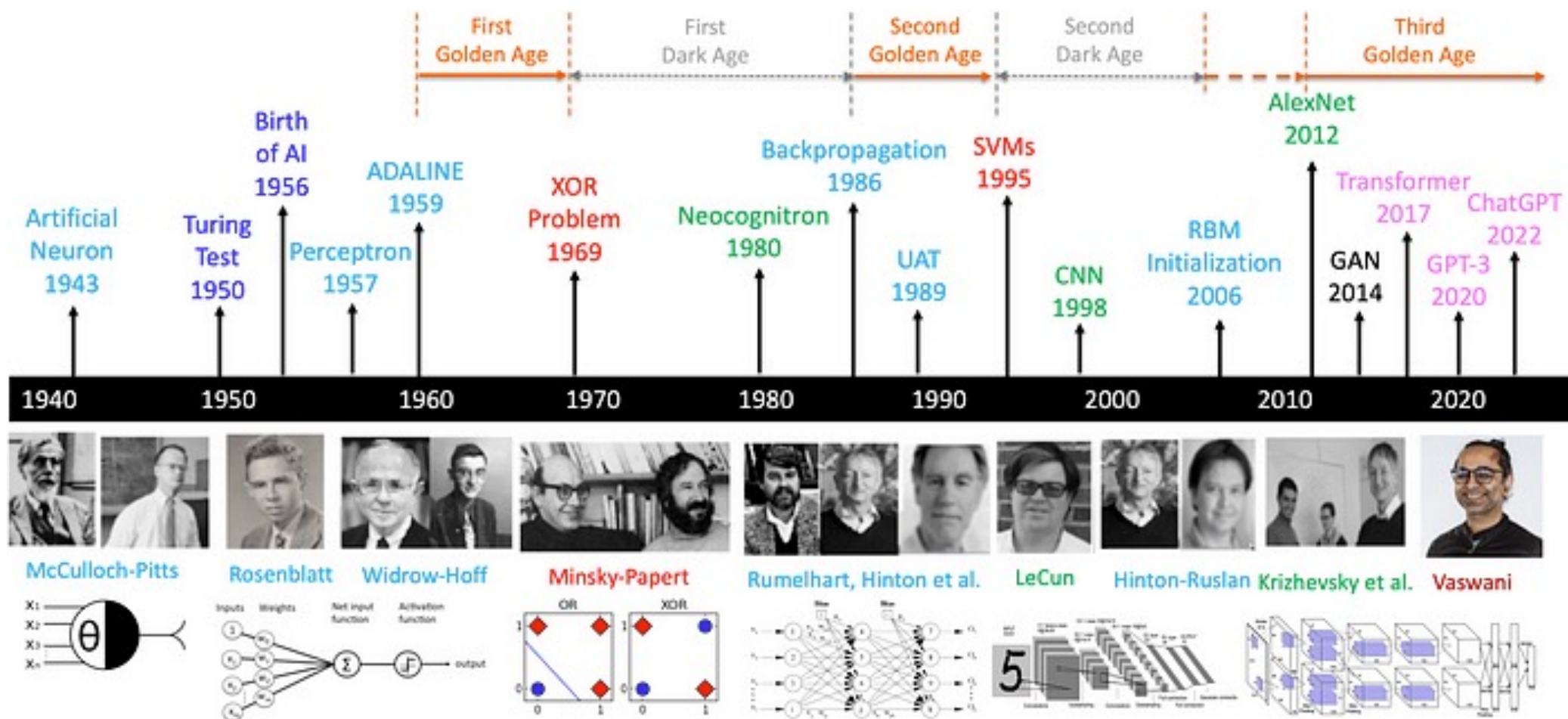
Machine Learning the
*"field of study that gives
computers the ability to
learn without being
explicitly programmed".*

History of Artificial Intelligence: A Timeline

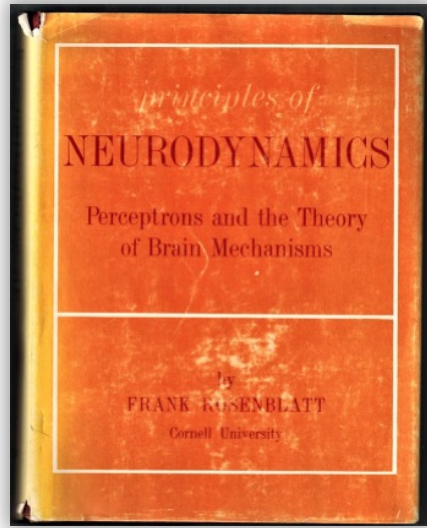
Def: AI Winter – a stagnant, quiet period for research and development of AI due to lack of funding from governments and businesses, and customer interest



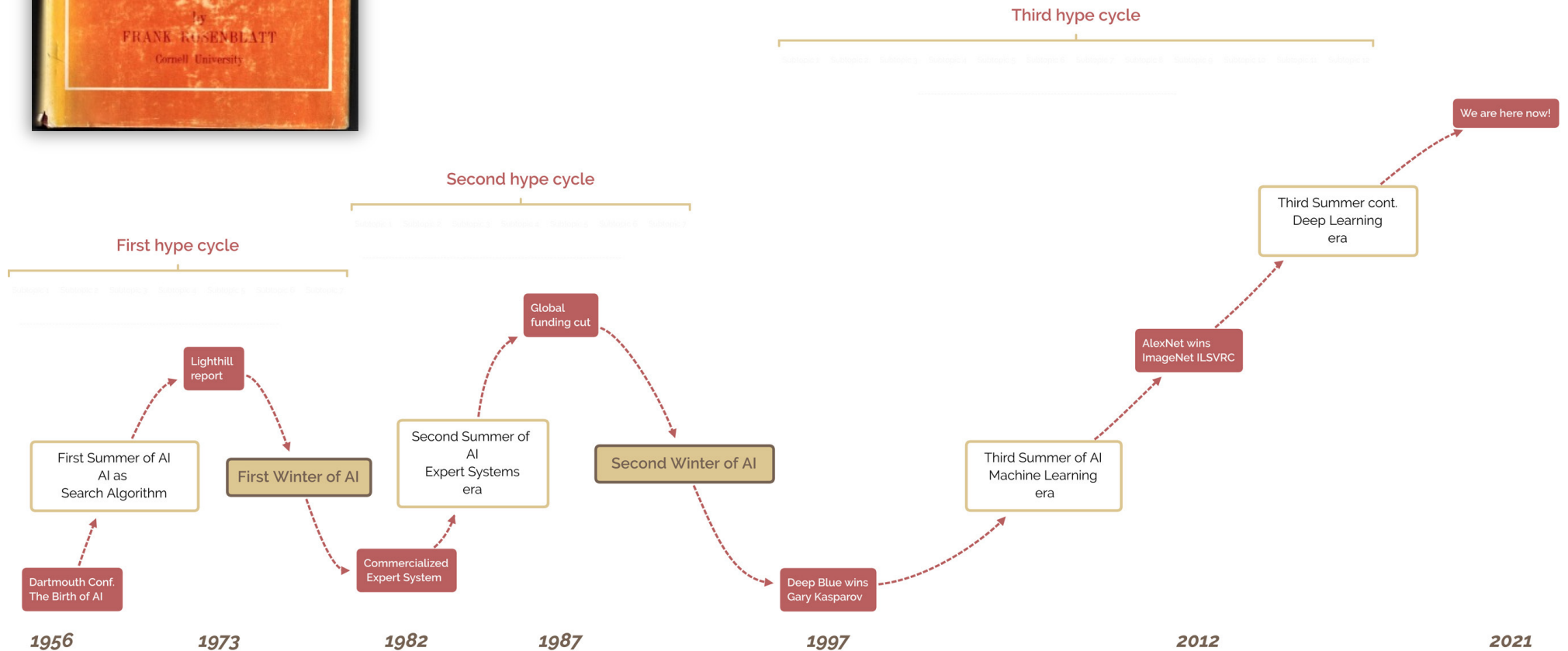
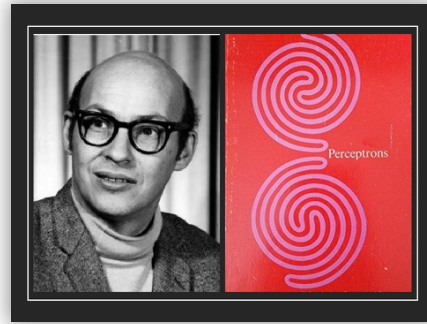
History of Artificial Intelligence: A (better) Timeline



History of Artificial Intelligence: A(nother) Timeline

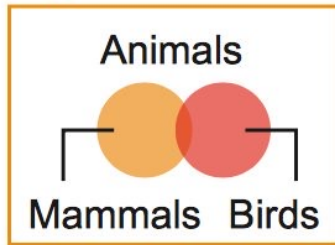


VS.



History of Artificial Intelligence: Five Tribes

Symbolists



Use symbols, rules, and logic to represent knowledge and draw logical inference

Favored algorithm

Rules and decision trees

Bayesians

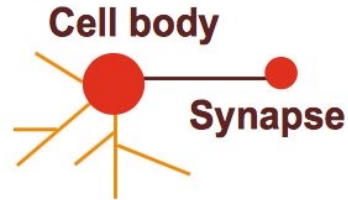


Assess the likelihood of occurrence for probabilistic inference

Favored algorithm

Naive Bayes or Markov

Connectionists



Recognize and generalize patterns dynamically with matrices of probabilistic, weighted neurons

Favored algorithm

Neural networks

Evolutionaries

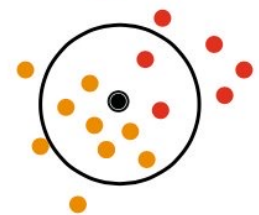


Generate variations and then assess the fitness of each for a given purpose

Favored algorithm

Genetic programs

Analogizers



Optimize a function in light of constraints ("going as high as you can while staying on the road")

Favored algorithm

Support vectors

Neural Networks: Structure of an Bio-NN

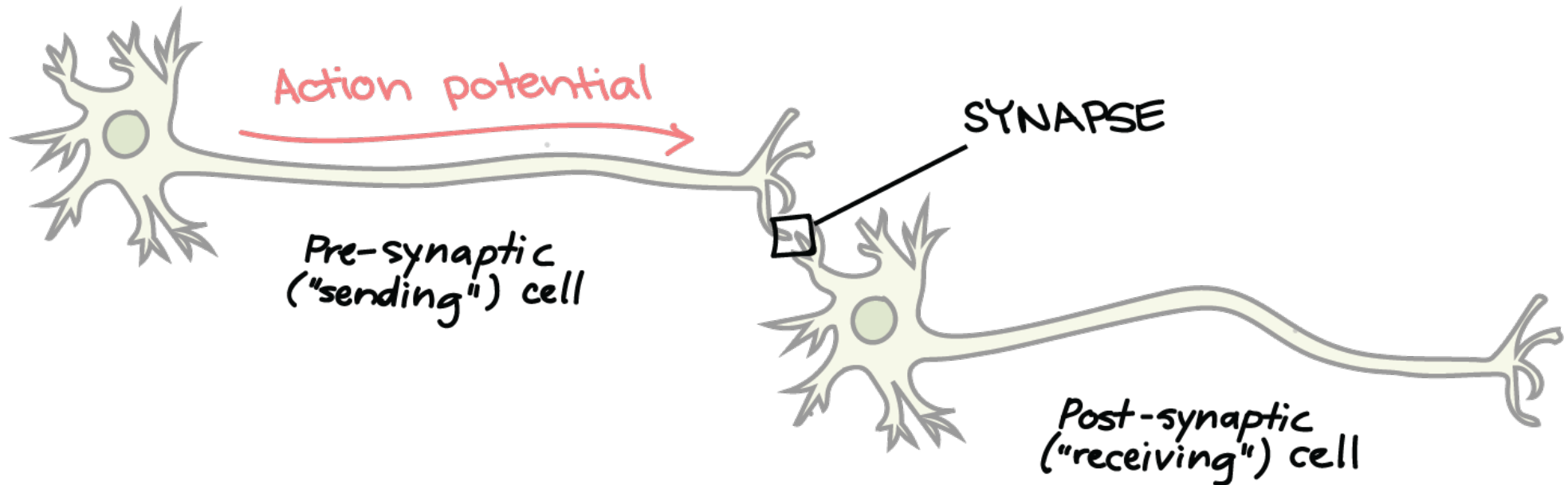
It's not just one, but many neurons that must be combined/processed (combining the neuron's **memory** with the **stimulation** sent to it)

to then determine if the **system** should:

- activate the neuron in the next layer

OR

- trigger a final response, or not



Neural Networks: Structure of an Bio-NN

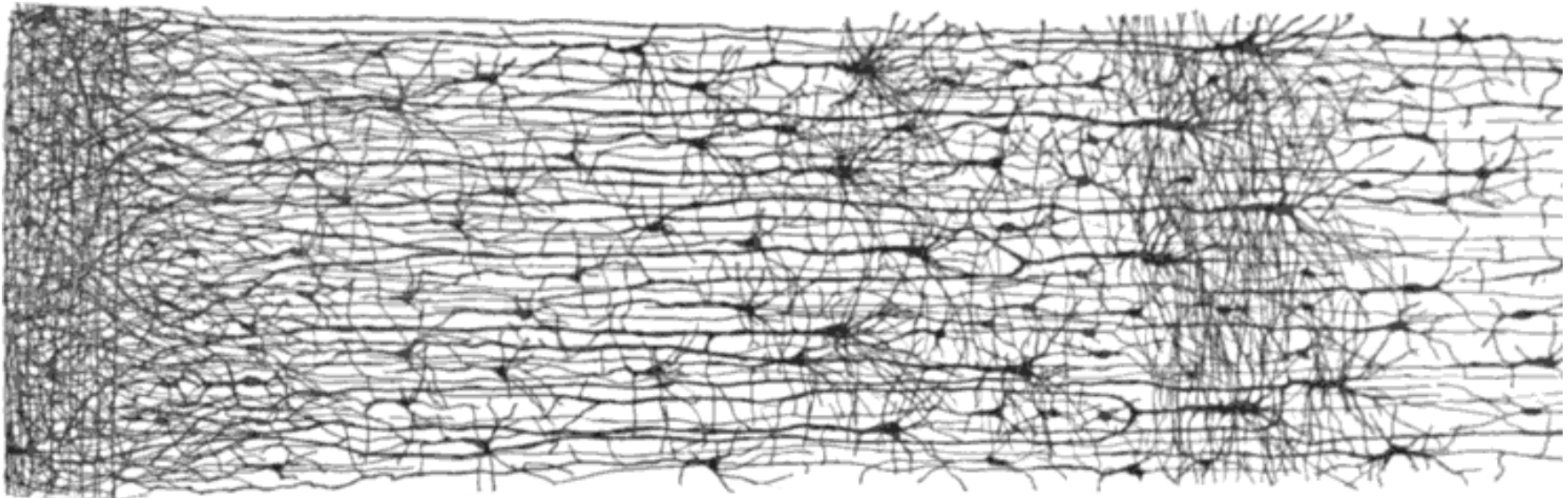
It's not just one, but many neurons that must be combined/processed
(combining the neuron's **memory** with the **stimulation** sent to it)

to then determine if the **system** should:

- activate the neuron in the next layer

OR

- trigger a final response, or not



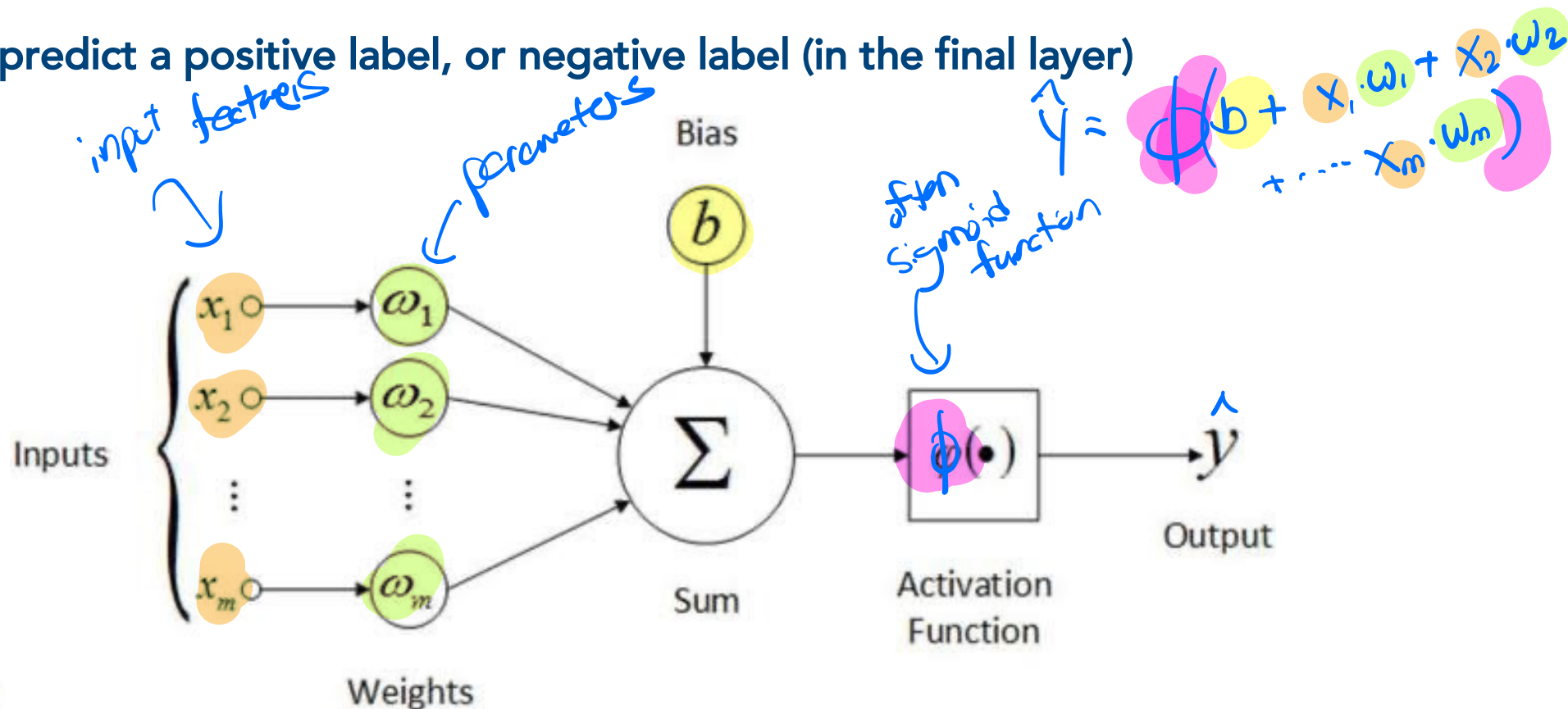
Neural Networks: Structure of an ANN

It's not just one, but many neurons that must be combined/processed (combining the neuron's **weight parameter** with the **x input** sent to it) to then determine if the **model** should:

- activate the neuron in the next layer (in a multi-layer neural network)

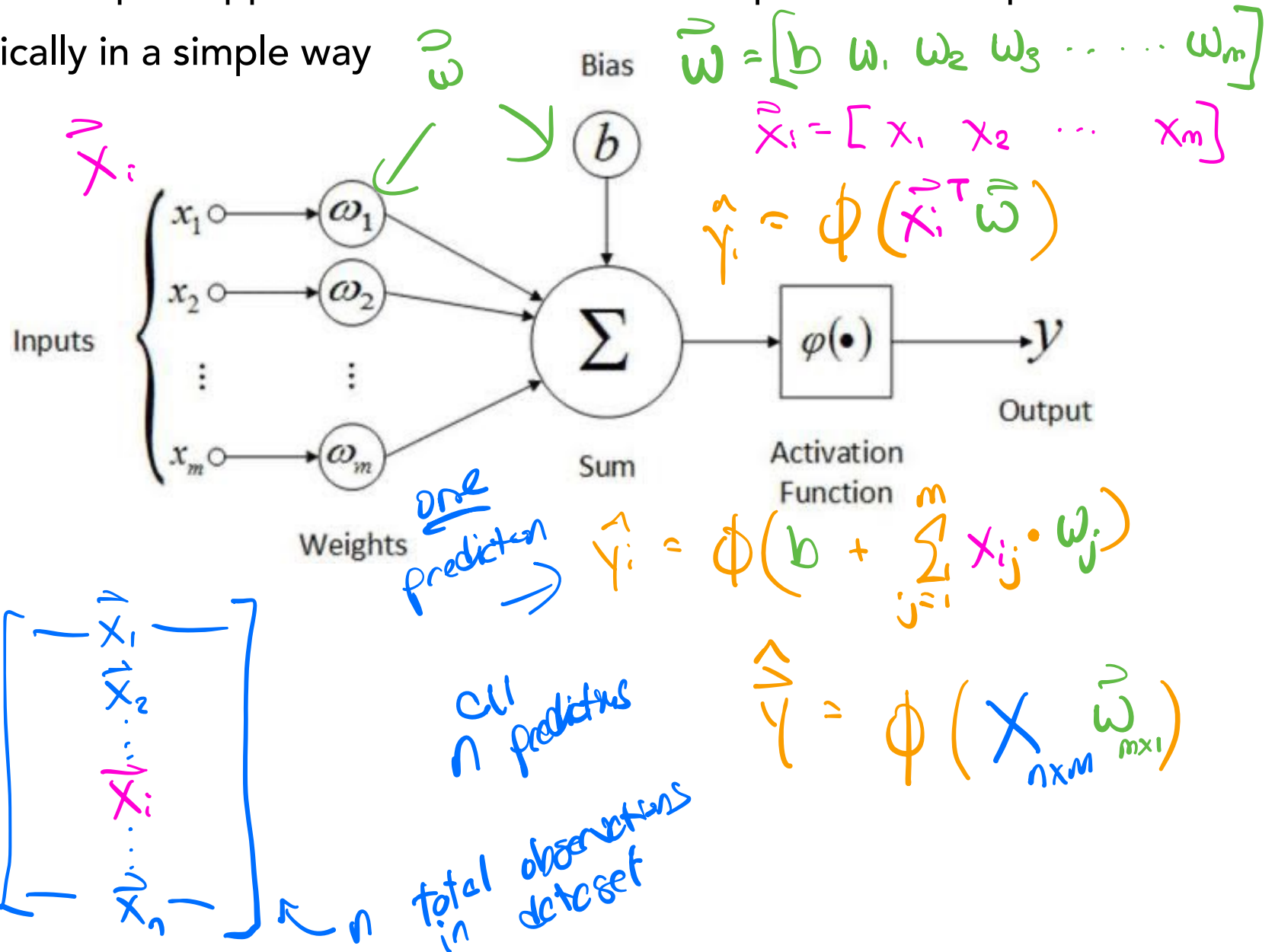
OR

- predict a positive label, or negative label (in the final layer)



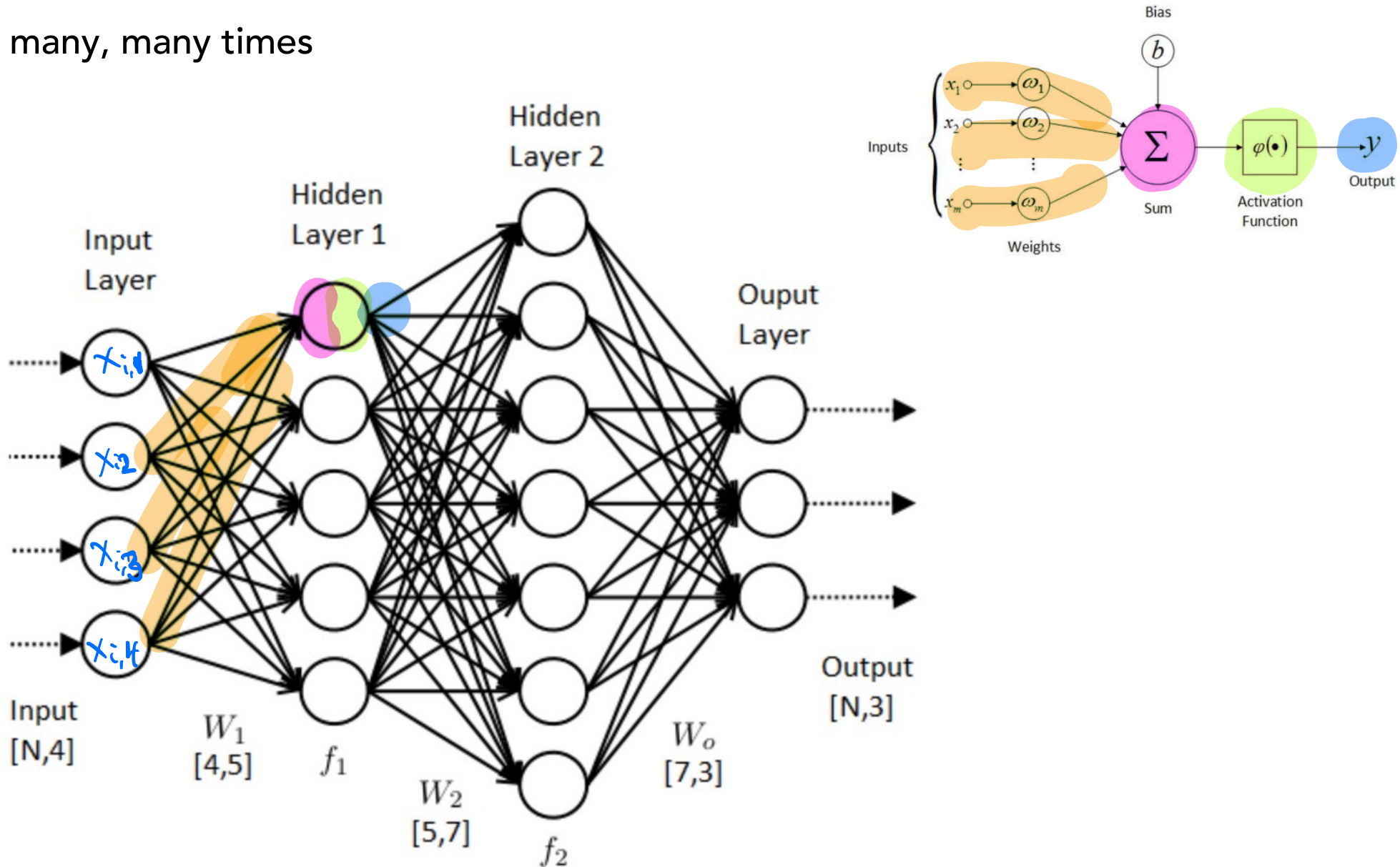
Neural Networks: Structure of an ANN

Despite the complex appearance and structure, it's possible to express mathematically in a simple way



Neural Networks: Structure of an ANN

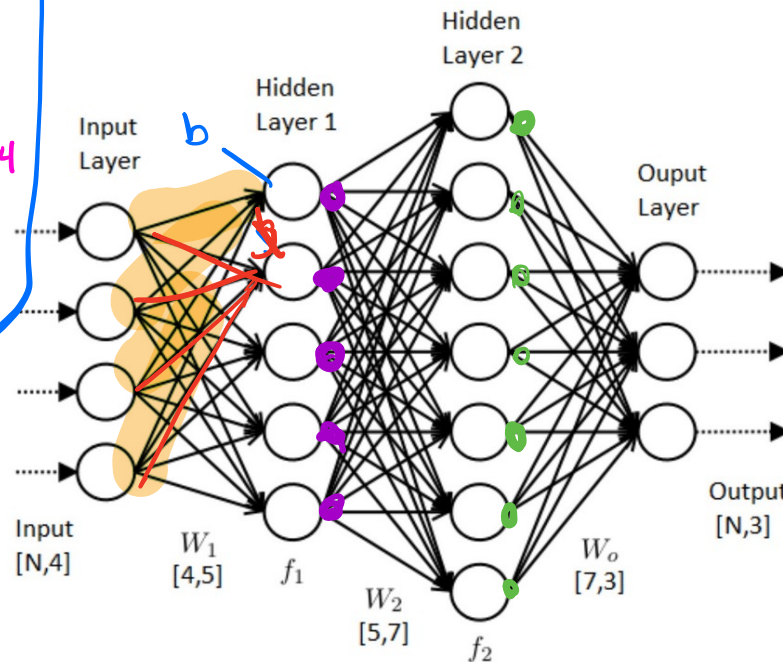
Multi-layer networks behave the same, just with this same model repeated many, many times



Neural Networks: Structure of a ANN

Q: How many parameters are in this neural network model?

$$X = \begin{bmatrix} x_{1,1} & x_{1,2} & x_{1,3} & x_{1,4} \\ \vdots & \vdots & \vdots & \vdots \\ x_{i,1} & x_{i,2} & x_{i,3} & x_{i,4} \\ \vdots & \vdots & \vdots & \vdots \\ x_{n,1} & x_{n,2} & x_{n,3} & x_{n,4} \end{bmatrix}$$



hand
row
n.p.p.

$$\hat{Y}_{n \times 3} = \begin{bmatrix} 0.3 & 0.6 & 0.1 \end{bmatrix}$$

activation
functions
(non-linear)

$$\phi^{(1)} \left(X_{n \times 4} \cdot \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \\ w_{41} \end{bmatrix} \right) = X_{n \times 5}^{(1)}$$

4×5

$$X_{n \times 5}^{(1)} \rightarrow \phi^{(2)} \left(X_{n \times 5}^{(1)} \cdot W_{5 \times 7}^{(2)} \right) = X_{n \times 7}^{(2)}$$

$$\phi^{(3)} \left(X_{n \times 7}^{(2)} \cdot W_{7 \times 3}^{(3)} \right) = \hat{Y}_{n \times 3}$$

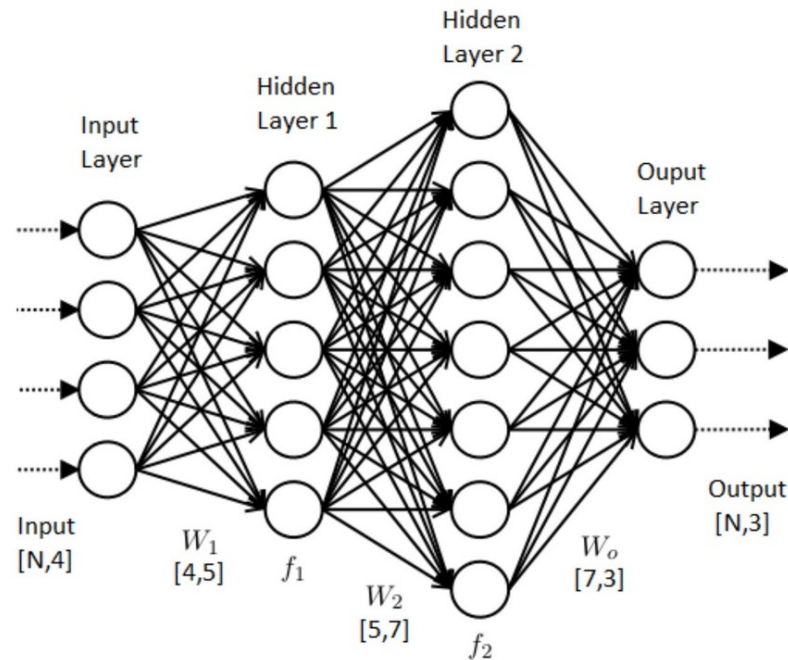
$$y_{n \times 7} = \phi \left(\phi \left(\phi \left(x \cdot w^{(1)} \right) \cdot w^{(2)} \right) \cdot w^{(3)} \right)$$

Neural Networks: Activation Functions

Q: Why do we need activation functions?

(i.e. what happens if we don't use them between layers?)

$$\begin{aligned} a \cdot b \cdot c \\ &= (a \cdot b) \cdot c \\ &= a(b \cdot c) \\ &= a \cdot c \cdot b \end{aligned}$$



$$\hat{y} = \left(\left(x \cdot w_{4 \times 5}^{(1)} \right) \cdot w_{5 \times 7}^{(2)} \right) w_{7 \times 3}^{(3)}$$

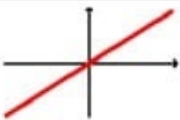
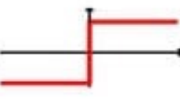
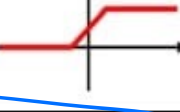
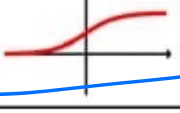
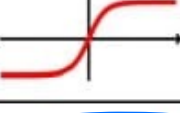
$$x \left(w^{(1)} w^{(2)} w^{(3)} \right)$$

$$w_{4 \times 3}$$

Neural Networks: Activation Functions

Activation functions are important because they allow for non-linearities, which are able to learn complex patterns/relationships.

(without them a neural network could be stated as a simple linear model)

Activation Function	Equation	Example	1D Graph
Linear	$\phi(z) = z$	Adaline, linear regression	
Unit Step (Heaviside Function)	$\phi(z) = \begin{cases} 0 & z < 0 \\ 0.5 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Sign (signum)	$\phi(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$	Perceptron variant	
Piece-wise Linear	$\phi(z) = \begin{cases} 0 & z \leq -1/2 \\ z + 1/2 & -1/2 \leq z \leq 1/2 \\ 1 & z \geq 1/2 \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multilayer NN	
Hyperbolic Tangent (tanh)	$\phi(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	Multilayer NN, RNNs	
ReLU	$\phi(z) = \begin{cases} 0 & z < 0 \\ z & z > 0 \end{cases}$	Multilayer NN, CNNs	

Neural Networks: A Simple Perceptron Model

Suppose we want to build a small, simple, neural network with one perceptron.

Q: How do we learn whether to 'activate' the neuron (based on some input, x)?

Neural Networks: A Simple Perceptron Model

Suppose we want to build a small, simple, neural network with one perceptron.

Q: How do we learn whether to 'activate' the neuron (based on some input, x)?

Specifically:

Q: What does the neural network look like?

Q: How do we train/fit this model? What loss function to use? etc.

Neural Networks: Structure of a ANN

Q: How many parameters would be in the neural network for the Airline Tweet dataset?

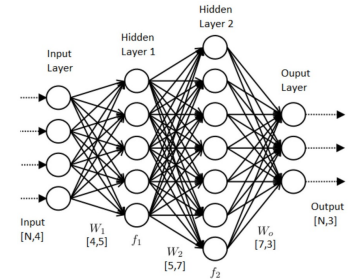
```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
X_train = tfidf_vectorizer.fit_transform(df['textclean']).toarray()
#X_train = tfidf_vectorizer.fit_transform(df['text']).toarray() # original tweet text (without our manual tokenization)

print(f"X_train.shape = {X_train.shape}")
type(X_train)
```

✓ 0.1s

X_train.shape = (8500 , 7480)

$$\begin{aligned} \phi^{(1)} \left(X_{n \times 7480} \cdot W_{7480 \times 5}^{(1)} \right) &\Rightarrow X^{(1)}_{n \times 5} \\ \phi^{(2)} \left(X^{(1)} \cdot W_{5 \times 7}^{(2)} \right) &\Rightarrow X^{(2)}_{n \times 7} \\ \phi^{(3)} \left(X^{(2)} \cdot W_{7 \times 3}^{(3)} \right) &\Rightarrow \hat{Y}_{n \times 3} \end{aligned}$$



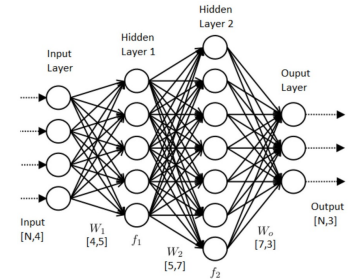
Neural Networks: Structure of a ANN

Q: How many parameters would be in the neural network for the Airline Tweet dataset if there were 2x as many tweets and from many different regions (with different dialects/languages)?

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
X_train = tfidf_vectorizer.fit_transform(df['textclean']).toarray()
#X_train = tfidf_vectorizer.fit_transform(df['text']).toarray() # original tweet text (without our manual tokenization)

print(f"X_train.shape = {X_train.shape}")
type(X_train)
```

✓ 0.1s



`X_train.shape = ( , )`

Software Modules/Tools for Coding ANNs

Software Modules/Tools: numpy

numpy is package for scientific/numerical computation w/ Python

Primary object numpy allows you to use is a numpy array (called an **ndarray**)

Some differences between numpy arrays and Python lists:

- fixed size at creation, changing size of an ndarray will create a new array and delete the original
- elements generally required to be of same data type
- mathematical operations are executed more efficiently and with less code than is possible using Python's built-in sequences
- many other packages rely on numpy (even though they may accept Python list data types as input, often converted to numpy objects for efficiency)

alist = [4, 1.7, 'abc']

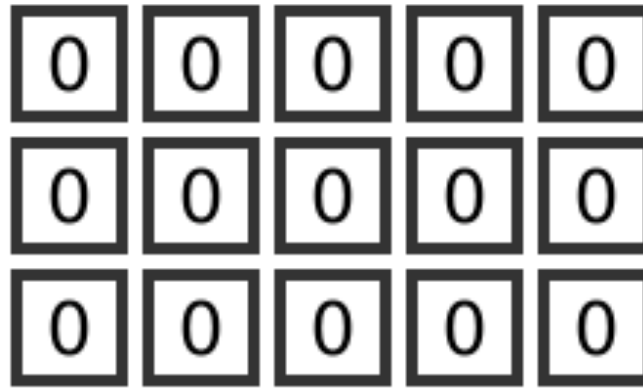
Software Modules/Tools: numpy

Rank 1 array:

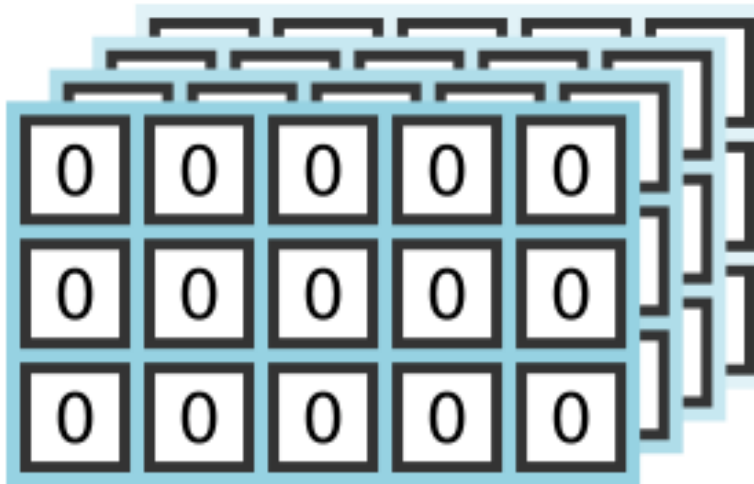


Rank 2 array:
(i.e. a matrix)

*rows &
cols*



Rank 3 array:



*rank 0
object is
a single
scalar number*

a = 3

Software Modules/Tools: numpy

Creating a ^{rank-1} 1-dimensional ndarray:

➤ `a = np.array([1, 2, 3, 4, 5, 6])`

a.shape → (6,)
a.size → 6

^{rank-2} A 2-dimensional ndarray:

➤ `a2 = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])`

a2.shape → (3, 4)

^{rank-3} 3-dimensional ndarray:

➤ `a3 = np.array([[[0, 1, 2, 3], [4, 5, 6, 7]],
 [[0, 1, 2, 3], [4, 5, 6, 7]],
 [[0, 1, 2, 3], [4, 5, 6, 7]]])`

a2.size = 12

a3.shape
→ (3, 2, 4)

a.size - total number of elements in the object

a.shape - tuple of number of elements in each dimension

a.ndim - number of dimension (aka rank)

Software Modules/Tools: numpy

Python lists

vs

numpy

```
>>> x1 = [1, 2, 3]
```

```
# what does this produce?
```

```
>>> x1 * 2
```

$[1, 2, 3, 1, 2, 3]$

```
>>> import numpy as np
```

```
>>> x1 = np.array([1, 2, 3])
```

```
# what does this produce?
```

```
>>> x1 * 2
```

$= [2, 4, 6]$
" 1×2 2×2 3×2

Software Modules/Tools: numpy

Python lists

vs

numpy

```
>>> x1 = [1, 2, 3]
>>> x2 = [1, 2, 3]

# what does this produce?
>>> x1 * x2
```

```
>>> import numpy as np
>>> x1 = np.array([1, 2, 3])
>>> x2 = np.array([1, 2, 3])

# what does this produce?
>>> x1 * x2
```

Software Modules/Tools: numpy

Python lists

vs

numpy

$x1.shape = (3,)$
 $x2.shape = (3,)$

```
>>> x1 = [1,2,3]
```

```
>>> x2 = [1,2,3]
```

```
# what does this produce?
```

```
>>> x1 * x2
```

```
Traceback (most recent  
call last):
```

```
  File "<stdin>", line 1,  
    in <module>
```

```
TypeError: can't multiply  
sequence by non-int of  
type 'list'
```

```
>>> import numpy as np
```

```
>>> x1 = np.array([1, 2, 3])
```

```
>>> x2 = np.array([1, 2, 3])
```

```
# what does this produce?
```

```
>>> x1 * x2
```

```
array([1, 4, 9])
```

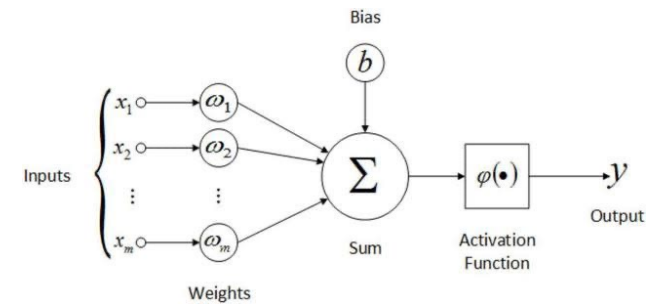
```
# can we take a dot product?
```

$np.dot(x1, x2)$

$\rightarrow 1 \times 1 + 2 \times 2 + 3 \times 3$

Software Modules/Tools: numpy

Assume a SLP for m-dimensional observations (m=3):



```
>>> import numpy as np
>>> x = np.array([[1, 4, 2, 7], [1, 3, 3, 5], [1, 4, 0, 2], [1, 2, 2, 8], [1, 0, 5, 5]])
>>> w = np.array([8, 2, -1, 3])
```

Q: What are size, shape, and ndim for x and w?

$x.shape \rightarrow (5, 4)$

$X_{5 \times 4}$

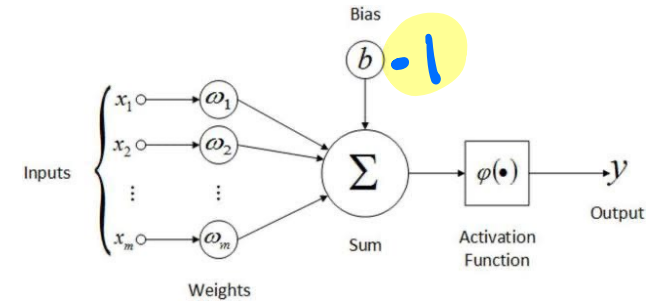
$w.shape \rightarrow (4,)$

$\vec{w}_{4 \times 1}$

$X \vec{w} =$
 $5 \times 4 \quad 4 \times 1$

Software Modules/Tools: numpy

Assume a SLP for m-dimensional observations:



```
>>> import numpy as np
>>> x = np.array([[1, 4, 2, 7], [1, 3, 3, 5], [1, 4, 0, 2], [1, 2, 2, 8], [1, 0, 5, 5]])
>>> w = np.array([8, 2, -1, 3])
```

Q: What are size, shape, and ndim for x and w?

```
>>> x.size
```

20

```
>>> x.shape
```

(5, 4)

```
>>> x.ndim
```

2

← rank

```
>>> w.size
```

4

```
>>> w.shape
```

(4,)

```
>>> w.ndim
```

1

← rank

$$X \cdot \vec{w}$$

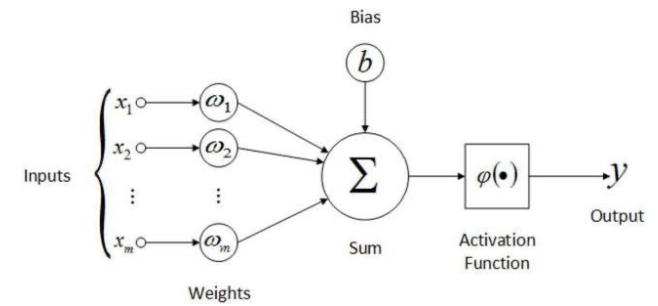
$n \times 4 \quad 4 \times 1$

$$\vec{w} = [8 \ 2 \ -1 \ 3]$$

Q: What are the bias, w1, w2, w3 parameters for this (already defined/trained) model?

Software Modules/Tools: numpy

Assume a SLP for m-dimensional observations:



```
>>> import numpy as np
>>> x = np.array([[1, 4, 2, 7], [1, 3, 3, 5], [1, 4, 0, 2], [1, 2, 2, 8], [1, 0, 5, 5]])
>>> w = np.array([8, 2, -1, 3])
```

Q: What does this produce?

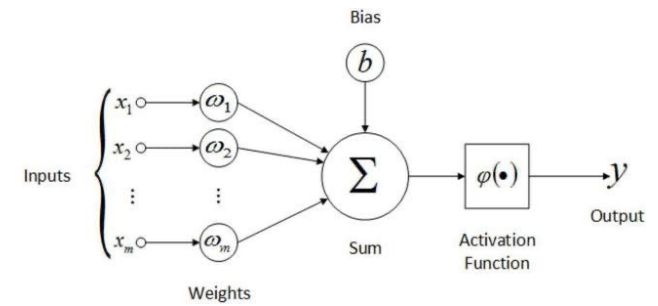
```
>>> x * w
```

$$X_{5 \times 4} \quad W_{4 \times 1}$$

$$X.\text{shape} = (5, 4)$$
$$W.\text{shape} = (4,)$$

Software Modules/Tools: numpy

Assume a SLP for m-dimensional observations:



```
>>> import numpy as np
>>> x = np.array([[1, 4, 2, 7], [1, 3, 3, 5], [1, 4, 0, 2], [1, 2, 2, 8], [1, 0, 5, 5]])
>>> w = np.array([8, 2, -1, 3])
```

Handwritten blue arrows pointing from the weights 8, 2, -1, 3 to w_0, w_1, w_2, w_3 respectively.

Q: What does this produce?

```
>>> x * w
```

```
array([[ 8,  8, -2, 21],
       [ 8,  6, -3, 15],
       [ 8,  8,  0,  6],
       [ 8,  4, -2, 24],
       [ 8,  0, -5, 15]])
```

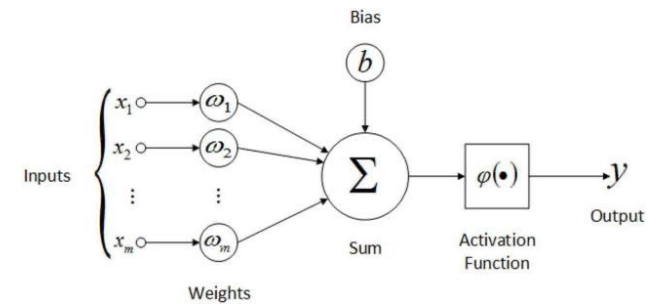
Handwritten blue arrow pointing to the result array with the text 5×4 .

Q: What does this produce?

```
>>> y_lin_pred = np.sum(x * w, axis=1)
```

Software Modules/Tools: numpy

Assume a SLP for m-dimensional observations:



```
>>> import numpy as np
>>> x = np.array([[1, 4, 2, 7], [1, 3, 3, 5], [1, 4, 0, 2], [1, 2, 2, 8], [1, 0, 5, 5]])
>>> w = np.array([8, 2, -1, 3])
```

Q: What does this produce?

```
>>> x * w
```

```
array([[ 8,  8, -2, 21],
       [ 8,  6, -3, 15],
       [ 8,  8,  0,  6],
       [ 8,  4, -2, 24],
       [ 8,  0, -5, 15]])
```

Q: What does this produce?

```
>>> y_lin_pred = np.sum(x * w, axis=1)
```

```
array([35, 26, 22, 34, 18])
```

$\nearrow \times 5$

$\phi^{(i)}(x^{(i)}) \Rightarrow \hat{y}$

Software Modules/Tools: numpy

Scalars vs vectors vs matrices vs

```
>>> s = np.array(3)
```

```
>>> s
```

```
array(3)
```

```
>>> x = np.array([1,2])
```

```
>>> x
```

```
array([1, 2])
```

```
# Q: What are size, shape, ndim (i.e. rank) of s and x?
```

Software Modules/Tools: numpy

Scalars vs vectors vs matrices vs

```
>>> s = np.array(3)
```

```
>>> s
```

```
array(3)
```

```
>>> x = np.array([1,2])
```

```
>>> x
```

```
array([1, 2])
```

Q: What are size, shape, ndim (i.e. rank) of s and x?

```
>>> s.size
```

```
1
```

```
>>> s.shape
```

```
()
```

```
>>> s.ndim
```

```
0
```

```
>>> x.size
```

```
2
```

```
>>> x.shape
```

```
(2,)
```

```
>>> x.ndim
```

```
1
```

rank 0

rank 1

Software Modules/Tools: numpy

Scalars vs vectors vs matrices vs

```
>>> s = np.array(3)
```

```
>>> s
```

```
array(3)
```

```
>>> x = np.array([1,2])
```

```
>>> x
```

```
array([1, 2])
```

```
# Q: What about size, shape, ndim of x * s?
```

Software Modules/Tools: numpy

Scalars vs vectors vs matrices vs

```
>>> s = np.array(3)
```

```
>>> s
```

```
array(3)
```

```
>>> x = np.array([1,2])
```

```
>>> x
```

```
array([1, 2])
```

```
# Q: What about size, shape, ndim of x * s?
```

```
>>> (x * s).size
```

```
2
```

```
>>> (x * s).shape
```

```
(2,)
```

```
>>> (x * s).ndim
```

```
1
```

```
>>> x * s
```

```
array([3, 6])
```

Software Modules/Tools: numpy

Scalars vs vectors vs matrices vs

```
>>> x = np.array([[1, 2],[10, 20],[100, 200]])
```

```
>>> x
```

```
array([[ 1,  2],  
       [10, 20],  
       [100, 200]])
```

$x.shape = (3, 2)$

```
>>> y = np.array((3,4))
```

```
>>> y
```

```
array([3, 4])
```

$y.shape = (2,)$

Q: What about size, shape, ndim of $x * y$?

Or x / y ?

Or $x - y$?

$x * y$

x / y

$x - y$

Software Modules/Tools: numpy

Scalars vs vectors vs matrices vs

```
>>> x = np.array([[1, 2],[10, 20],[100, 200]])
```

```
>>> x
```

```
array([[ 1,  2],
       [10, 20],
       [100, 200]])
```

```
>>> y = np.array((3,4))
```

```
>>> y
```

```
array([3, 4])
```

Q: What about size, shape, ndim of $x * y$? Or x / y ? Or $x - y$?

```
>>> x * y
```

```
array([[ 3,  8],
       [30, 80],
       [300, 800]])
```

$(3, 2)$

```
>>> x / y
```

```
array([[ 0.3333,  0.5  ],
       [ 3.3333,  5.   ],
       [33.3333, 50.   ]])
```

$(3, 2)$

```
>>> x - y
```

```
array([[ -2,  -2],
       [ 7,  16],
       [97, 196]])
```

$(3, 2)$

Software Modules/Tools: numpy

Suppose input to a model is a 256x256 pixel image w/ 3 channels (R, G, B)

```
>>> x.shape  
(256, 256, 3)
```

← rank 3

```
>>> x
```

```
array([[[ 0, 1, 2],  
       [ 3, 4, 5],  
       [ 6, 7, 8],  
       ...,  
       [ 759, 760, 761],  
       [ 762, 763, 764],  
       [ 765, 766, 767]],  
      [[ 768, 769, 770],  
       [ 771, 772, 773],  
       [ 774, 775, 776],  
       ...
```

```
# RGB values at pixel at location 4,4
```

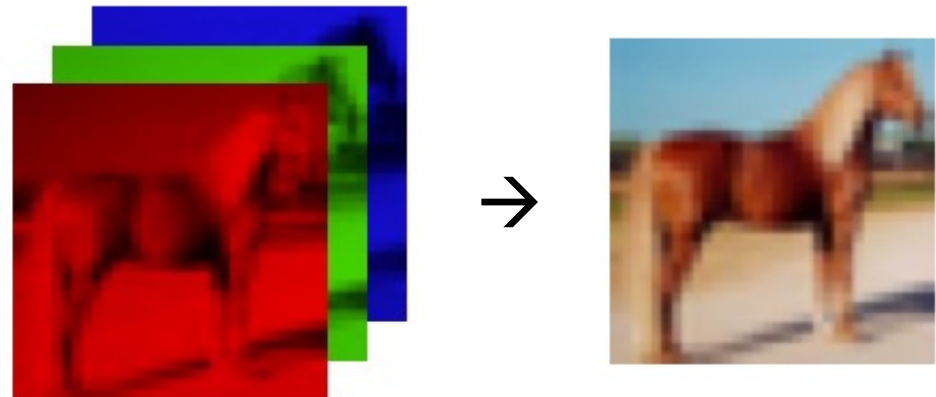
```
>>> x[4,4,]
```

```
array([3084, 3085, 3086])
```

Q: How to scale the image's R, B down by 50% and G up by 50?

```
>>> scale = np.array((
```

0.5, 1.5, 0.5))
↑ ↑ ↑
R G B



Software Modules/Tools: numpy

```
>>> x.shape
(256, 256, 3)
```

```
>>> x
array([[[ 0, 1, 2],
        [ 3, 4, 5],
        [ 6, 7, 8],
        ...,
        [759, 760, 761],
        [762, 763, 764],
        [765, 766, 767]],
       [[ 768, 769, 770],
        [ 771, 772, 773],
        [ 774, 775, 776],
        ...
```

```
# RGB values at pixel at location 4,4
```

```
>>> x[4,4,]
array([3084, 3085, 3086])
```

Q: How to scale the image's R, B down by 50% and G up by 50?

```
>>> scale = np.array((0.5, 0.5, 1.5))
```

```
>>> scale
array([0.5, 0.5, 1.5])
```

```
>>> scale.shape
(3,)
```

```
>>> x * scale
```

```
array([[0.000000e+00, 5.000000e-01, 3.000000e+00],
       [1.500000e+00, 2.000000e+00, 7.500000e+00],
       [3.000000e+00, 3.500000e+00, 1.200000e+01],
       ...,
       [3.795000e+02, 3.800000e+02, 1.141500e+03],
       [3.810000e+02, 3.815000e+02, 1.146000e+03],
       [3.825000e+02, 3.830000e+02, 1.150500e+03]],
       [[3.840000e+02, 3.845000e+02, 1.155000e+03],
       [3.855000e+02, 3.860000e+02, 1.159500e+03],
       [3.870000e+02, 3.875000e+02, 1.164000e+03],
       ...
```

```
>>> (a*s).shape
```

```
(256, 256, 3)
```

Software Modules/Tools: numpy

Q: Suppose we know have $n=500$ images, then what would be shape of x ?

```
>>> x.shape
```

Q: How to retrieve the 10th image/observation? (and what is its shape?)

```
>>> x[
```

Q: Can we scale all 500 observations at once using same scale array?

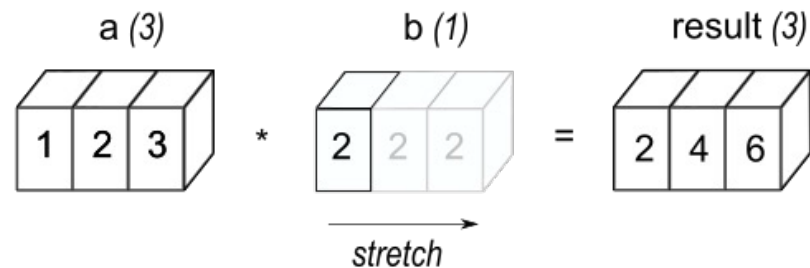
```
>>> x * scale    # will this work as expected? what will be the shape?
```

Software Modules/Tools: numpy Broadcasting

Broadcasting is a powerful feature of numpy (and PyTorch) to efficiently work with arrays of different shapes and ranks.

It also allows for trivial parallelization of multi-dimensional arrays!

Ex: numpy knows to 'grow' or 'stretch' shape of smaller object to be compatible with the other



```
>>> a = np.array([1.0, 2.0, 3.0])
>>> b = 2.0
>>> a * b
array([2., 4., 6.])
```

Software Modules/Tools: numpy Broadcasting

General broadcasting rules:

Compare shapes of two arrays in an element-wise manner, starting with the trailing (i.e. rightmost) dimensions and working to the left.

Two dimensions are compatible when:

- they are equal, or
- one of them is 1

$(256, 256, 3)$
 $256, 256, (3,)$

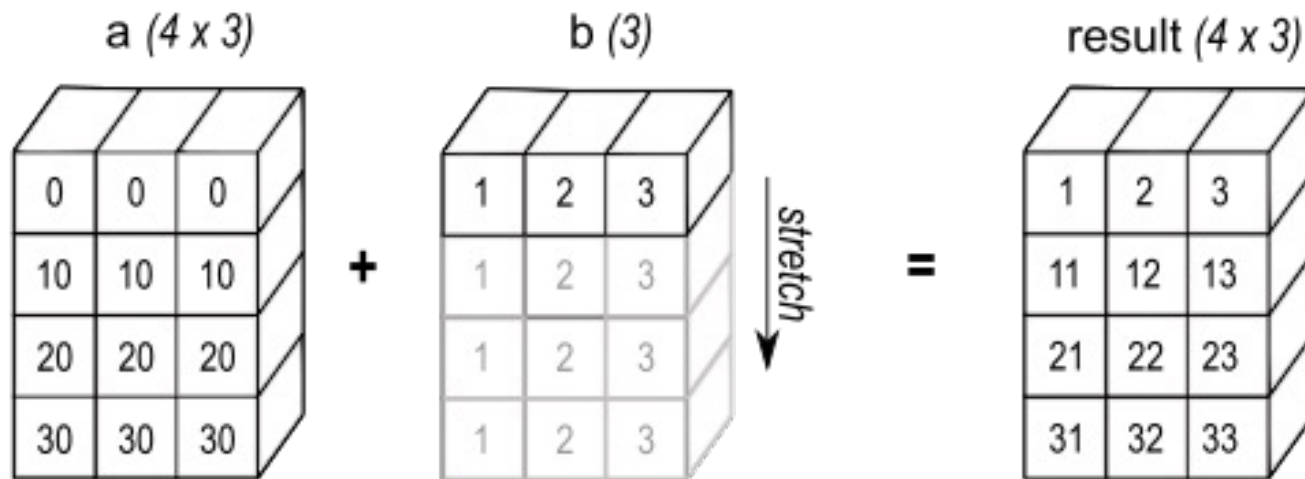
Software Modules/Tools: numpy Broadcasting

Ex: Stretching/growing a 1d array to be compatible with a 2d array

A (2d array) : 4 x 3

B (1d array) : 3

Result : 4 x 3



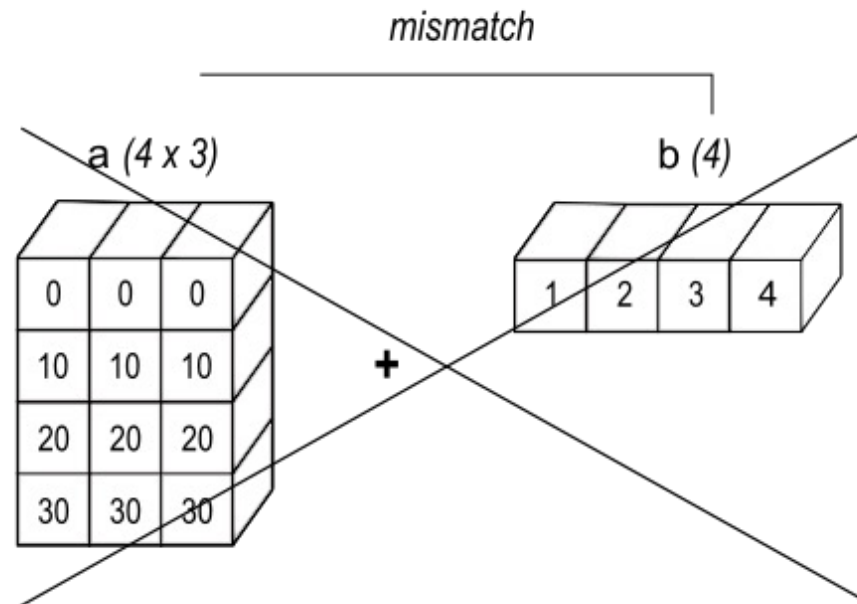
Software Modules/Tools: numpy Broadcasting

Ex: Arrays are incompatible when trailing dims don't match
(and are greater than 1)

A (2d array): 4 x 3

B (1d array): 4

not compatible



Software Modules/Tools: numpy Broadcasting

Q: Multiplying A and B, what is shape of the result?

A (2d array) : 5 x 4

B (1d array) : 1

Result (2d array) : x

A (2d array) : 5 x 4

B (1d array) : 4

Result (2d array) : x

A (3d array) : 15 x 3 x 5

B (3d array) : 15 x 1 x 5

Result (3d array) : x x

Software Modules/Tools: numpy Broadcasting

Q: Multiplying A and B, what is shape of the result?

A (3d array) : 15 x 3 x 5

B (2d array) : 3 x 5

Result (3d array) : x x

A (3d array) : 15 x 3 x 5

B (2d array) : 3 x 1

Result (3d array) : x x

Software Modules/Tools: numpy Broadcasting

Q: Multiplying A and B, what is shape of the result?

A (1d array) : 4

B (1d array) : 3

Result :

A (3d array) : 8 x 4 x 3

B (2d array) : 2 x 1

Result :

Software Modules/Tools: numpy Broadcasting

Q: What if we really want to combine A and B somehow? (e.g. outer product)

```
>>> a = np.array([0.0, 10.0, 20.0, 30.0])  
>>> b = np.array([1.0, 2.0, 3.0])  
>>> np.outer(a, b)
```

Q: Is there another way using broadcasting?

Software Modules/Tools: numpy Broadcasting

Q: What if we really want to combine A and B somehow? (e.g. outer product)

```
>>> a = np.array([0.0, 10.0, 20.0, 30.0])
>>> b = np.array([1.0, 2.0, 3.0])
>>> np.outer(a, b)
```

Q: Is there another way using broadcasting?

```
>>> (a[:, np.newaxis]).shape
(4, 1)
>>> (a[:, np.newaxis]) * b
array([[ 0,  0,  0],
       [10, 20, 30],
       [20, 40, 60],
       [30, 60, 90]])
```

Should AI Tools be used for Mental/Emotional Support?

Disc 2

Discussion 2: Should AI Chatbots be used as mental/emotional support tools?

[Eliza](#) , reportedly the world's first AI chatbot, was created in the 1960's and caused a lot of excitement and discussion. Its creator, Joseph Weizenbaum, even stepped away from AI research after he saw people relying on Eliza as if "she" were a real, qualified, human therapist (he even became one of the first major critics of AI).

ChatGPT launched in November of 2022, and with each improvement to the model it seems more and more human like. Some [people have reported using ChatGPT as a therapist](#)  for mental and emotional support.

[Character AI](#)  is a company that offers chatbots with pre-configured prompts (behind the scenes) for many use cases. Its customers often use the service as a way to have a chatbot boyfriend/girlfriend. Last year a [14-year old boy committed suicide](#)  shortly after messaging with his CharacterAI girlfriend (the case finally [ended a few weeks ago](#)  with the company paying a settlement to the family). Companies are beginning to [restrict access to people 18 years or older](#) . Even for adults, if these human-chatbot relationships don't end in catastrophe, they can still [take control of a person's time \(and money\)](#) .

Reply to this post with your yes/no answer to the question:

- "Should AI Chatbots be used as mental/emotional support tools?"

Explain why you answered "yes" or "no" by providing 2-3 supporting arguments. If you answered "yes", then state whether you think there should be any limits to how are they accessed/used. If you answered "no", then state whether you think there should be any exceptions/special cases where AI tools should be allowed to be used this way. Try to provide at least one reference to an article/source that supports your position.

Today

- ▶ Neural Networks
- ▶ Numpy and PyTorch
(Broadcasting and Optimization)
- ▶ Quiz 1

To Do

- ▶ Discussion 2 (Feb 7th)
- ▶ ~~Quiz 1 Retakes~~

Next Time

- ▶ Training a Neural Network
- ▶ PyTorch (Backpropagation and Gradient Descent)
- ▶ Lab 2



February 3, 2026

The MSU Denver Math Seminar
Presents

“Fixed Points and Fair Division”

Dr. JOHN CARTER

Monday, February 9, 2026
2:00 PM in Science 1015

Join Us for Pizza, Soda, and a Good Talk!

Free Books for All Students and Faculty!
(Many thanks to Louis Krupp for donating books to our math seminar!)

Abstract:

Imagine a rental house with three rooms where the rent is \$900 per month. Three people plan to rent the house together, but the rooms are wildly different. What is the fairest way to divide the rent? If you immediately said ‘Obviously \$300 each’ imagine one roommate paying for a glorified closet while the other gets a private bath and a kitchenette. Does it still seem fair? If not is there a mathematical solution? I will share one solution that surprisingly uses the discrete version of the Brower fixed point theorem. I’ll even pass on a link to a program that helps you solve such division problems.

Info about the speaker:

Dr. John Carter was born in Denver, later he joined the faculty at MSU Denver where he is a Professor and, some say, the chair of MAST. He studied Algebraic Topology at the University of Oregon and is always on the lookout for real world applications of topology.

<https://msudenver.enterprise.localist.com/event/2026-msu-denver-scholarship-fair>

2026 MSU Denver Scholarship Fair

Wednesday, February 11, 2026 10:00am to 2:00pm
MST

I'm Interested



DSML 4220: Day 3